Identity Access Management (IAM)

IAM 101

- Manage users and their level of access to the AWS Console (Global/Universal, not regional)
- Centralized: Control of your AWS Account
- Access: Shared access to your AWS account
- Permissions: Granular permissions
- Identity Federation: Supports well-known identity providers such as Active Directory, Facebook, and LinkedIn
- Allows MFA, Temporary Access, Setting Up Own Rotation Policies
- IAM Policy Simulator: Testing policies before committing to production and validation (Works for users, groups, and roles)
- New users are assigned an access key ID and secret access key for programmatic access (Secret Access Key can only be viewed and downloaded once)
- Can customize and create your own password rotation policies

Users, Groups, and Roles

- Users: End Users
- Groups: Collections of users under one set of permissions
- Roles: Can be assigned to users, applications, and services
- Policy: A document that defines one or more permissions and can be attached to a user, group, or role

EC2

EC2 101

- Elastic Compute Cloud (VM in the cloud)
- Can have servers up in minutes, pay for what you use, grow and shrink when needed

EC2 Pricing Options

- On Demand: Pay by the hour or second
- Reserved: Reserved capacity for 1-3 years, discount on hourly charge (predictable usage)
    - Standard Reserved
    - Convertible Reserved: Smaller discount but can change to a different reserved instance type of equal or greater value
    - Scheduled Reserved: Launch within time window you define

- Spot: Purchase unused capacity at a massive discount, prices fluctuate with supply and demand, can be interrupted if AWS needs the capacity back, great for applications with flexible start and end times
- Dedicated: Physical server, most expensive
    - Great for compliance/regulatory restrictions
- Savings Plans: Commit to 1 or 3 years to use a specific amount of compute power

EC2 Instance Types

- Determines the hardware of the instance, has different capabilities based on requirements of application
    - General Purpose
    - Compute Optimized
    - Accelerated Computing
    - Storage Optimized

EBS Volumes

- Elastic Block Store
- Types:
    - General Purpose SSD (gp2 and gp3): Boot disks and general applications
    - Provisioned IOPS SSD (io1 and io2): Latency sensitive applications, use this if you really need performance that gp does not provide (gp provides 16000 IOPS while this provides 64000 IOPS)
    - Provisioned IOPS SSD (io2 Block Express): Storage Area Network (SAN) in the cloud, highest performance, sub-millisecond latency: Largest, most critical high performance operations (256000 IOPS)
    - Throughput Optimized HDD (st1): Suitable for big data, data warehouses, ETL, can't be a boot volume
    - Cold HDD (sc1): Less frequently accessed data, can't be a boot volume
- IOPS: Measures number of read and write operations per second important for quick transactions, low latency
- Throughput: Number of bits read or written per second, important for large datasets, I/O sizes, and complex queries

EBS Snapshots

- Point in time copy of an EBS volume, great for backups or creating a new EBS volume
- Encrypted Snapshot creates encrypted volumes, unencrypted snapshots create unencrypted volumes

Elastic Load Balancer

- Distributes network traffic across a group of servers
- Application Load Balancer: HTTP and HTTPS (Operates at Layer 7)

- Network Load Balancer: TCP and high performance (Operates at Layers 4) Typically isn't used for websites, mostly servers, protocols, and systems that aren't web applications
- Classic Load Balancer: Legacy Option, supports both HTTP/HTTPS and TCP
- **Gateway Load Balancer**: Allows to load balance workloads for 3rd party virtual appliances running in AWS
- 7 Layer Model (OSI Model): Conceptual framework which describes the functions of a network
- X-Forwarded-For Header: Identify the originating IPv4 address of a client connecting through a load balancer
- 504 Gateway Timeout: Target failed to respond, check your application

Route 53

- Amazon's DNS Service, allows you to map a domain name that you own to an EC2 instance, Load Balancer, or S3 Bucket
- **Hosted Zone**: Container for DNS records for domain (Like a folder that contains all DNS rules for a domain like mywebsite.com, www.mywebsite.com, api.mywebsite.com)
- **A Record**: Allows you to route traffic to a resource using an IPv4 address (mywebsite.com -> 12.12.12.12)
- Alias: Allows you to route traffic to the top of the DNS namespace and send it to a resource within AWS (Like an A Record but without an IP and uses an AWS service like mywebsite.com -> ALB)

CLI

- Follow the principle of least privileged
- Use IAM groups

CLI Pagination

- Can control the number of items included in the output when you run a CLI command
- Could run into a timed out error, can adjust **—page-size** option to limit the amount of items from API calls

RDS 101

- Tables with rows (data items) and columns (fields)
- SQLServer, Oracle, MySQL, PostgreSQL, MariaDB, Amazon Aurora
- Generally used for Online Transaction Processing (OLTP) workloads
- Online Transaction Processing (OLTP) vs Online Analytics Processing (OLAP)
    - **OLTP** is about processing data from transactions in real time
    - OLAP is about processing complex queries to analyze historical data (data analysis), RDS is not suitable for analyzing large amounts of data, use something like Redshift instead

RDS Multi-AZ and Read Replicas

- Multi-AZ is an exact copy of your database in another availability zone, used for disaster recovery, if a failure occurs AWS will automatically switch over
- Read Replica is a read-only copy of your database, can be located anywhere (same or different availability zone and region), has its own endpoint, can be promoted to be independent database
    - Requires automatic backup

RDS Backups and Snapshots

- Database Snapshot: Point in time copy of storage volume attached to DB, not automated (manual), no retention period, no transaction logs
- Automated Backup: Enabled by default
- RDS Automated Backup: Can recover database to any point in time within retention period, stored in S3 for free
- Can't encrypt an encrypted RDS DB
    - Can take a snapshot, and encrypt the snapshot, and restore using snapshot

Increasing Scalability using RDS Proxy

- Basically is a connection cache
- Application pointed towards RDS Proxy, RDS Proxy pools and shares database connections for scalability and efficiency, then goes to the DB
- Proxy maintains active connections for performance
- Serverless and scales automatically, preserves application connections, detects failover and routes requests, deployable over Multi-AZ, up to 66% failover times

Elasticache 101

- In-memory cache (Key Value) to improve database performance, great for **read heavy** database workloads and when data is not prone to frequent changes
- Types:
    - Memcached: **Basic** object caching, no persistence, Multi-AZ or failover
    - Redis: More **sophisticated** solution and supports sorting and ranking data for complex data types
- Doesn't help with heavy write loads and OLAP Queries

MemoryDB For Redis

- Massively scalable in-memory database, highly available, multi-AZ, can be used as a primary DB with fast performance
- **MemoryDB** vs ElastiCache:
    - ElastiCache for Redis: In-memory DB cache and is fast but not ultra-fast
    - MemoryDB for Redis: Removes the need for a DB plus cache and has ultra fast performance

Parameter Store

- Stores configuration data and less-sensitive secrets (like environment variables, log levels, etc.)
- Supports plain text and encrypted values using KMS
- No automatic rotation; updates are managed manually
- Integrated with EC2, Lambda, and other AWS services for retrieving parameters

Secrets Manager

- Stores and manages sensitive credentials (like database passwords or API tokens)
- Encrypts all secrets with KMS and supports automatic rotation, automatic rotation can work on things related to AWS services
- Provides built-in auditing and fine-grained access control
- Best for highly sensitive data that requires rotation and lifecycle management

EC2 Image Builder

- Can validate your images using AWS provided tests or custom tests
- Automates the process of creating and maintaining your images and can share your AMIs with other accounts
- Image **Pipeline**: Defines the configuration and end-to-end process of building images
- Image **Recipe**: Image builder creates a recipe for each image which can be shared, version controlled, and re-used (Contains Source Image and Build Components)
- Build Components: Software components included in the image
- Steps:
    - 1. Base OS: Provide a base OS Image (like Amazon Linux 2 AMI)
    - 2. Define software to Install (.Net, Node.js, Python, latest security updates, latest kernel, security settings)
    - 3. Run test on new image
    - 4. Distribute the image to regions

AMIs in Different Regions

- AMI only exists in a **single** Region, you can only use it in your region you specify, to use in another region you'll need to create a copy
- Encryption stays, and you can't remove encryption in the copying process, but you can add encryption

Question

1. Individual EC2 instances are provisioned _____.
   Select one answer option below.
   A. In Regions
   B. To a User

C. Globally
D. In Availability Zones

S3

S3 101

- Simple Storage Service
- Provides secure, durable, highly-scable object storage
- Simple to use
- Scalable: Allows you to store and retrieve any amount of data at low cost
- Manages data as objects rather than in file systems or data blocks (can store any type of file)
- Features:
    - Unlimited Storage
    - Objects up to 5TB in size
    - Stored in buckets (similar to folders)
        - Naming of buckets must be globally unique
        - bucket-name.s3.region.amazonaws.com/key-name
- Key-Value Store:
    - Key: Name of the object (Ralphie.jpg)
    - Value: Data itself
    - Version ID
    - Metadata: Data about the data you're storing (content-type, last-modified, etc.)
- Highly available and Highly Durable (Being stored safely and not being lost or corrupted)
- Characteristics:
    - Tiered Storage
    - Lifecycle Management: Rules to automatically transition objects to a cheaper storage tier or delete objects that are no longer required after a set amount of time
    - Versioning
- Securing data:
    - Server-Side Encryption
    - Access Control Lists (ACLs): Define which AWS accounts or groups are granted access
    - Bucket Policies: Specify what actions are allowed or denied

S3 Storage Classes

- All have >=3 Availability Zones except for S3 One Zone-IA
- S3 Standard:
    - High Availability and Durability
    - Frequent Access
    - Suitable for Most workloads

- S3 Standard-Infrequent Access (**S3-IA**):
    - Data that is accessed less frequently but requires rapid access
    - Pay to access the data
    - Great for long-term storage, backups, and disaster recovery files (minimum storage duration: 30 days)
- S3 **One Zone**-Infrequent Access:
    - Like S3-IA, but data is stored redundantly within a single Availability Zone
    - Costs 20% less than S3-IA
    - Non-critical data
- S3 **Glacier Instant Retrieval, S3 Glacier Flexible Retrieval, and S3 Glacier Deep Archive**:
    - Very infrequently accessed
    - Pay each time you access your data
    - Use only for archiving data
        - Glacier Instant Retrieval: Data that is accessed once per quarter and quick retrieval times
        - Glacier Flexible Retrieval: Data that is accessed occasionally and takes hours or minutes to retrieve
        - Minimum storage duration: 90 days
        - Glacier Deep Archive: Rarely accessed data, 12 hour retrieval time, minimum storage is 180 days
- S3 Intelligent Tiering:
    - 2 tiers: Frequent and Infrequent access
    - Automatically moves your data to the most cost-effective tier based on how frequency you access each object
- Relative Costs:
    - S3 Standard has the highest cost
    - Intelligent-Tiering is cost-optimized for unknown access patterns
    - Infrequent Access and Glacier: Retrieval fees apply

Securing S3 Buckets

- By default, buckets are private (no public access), and only the bucket owner can do anything
- Bucket Policies: Permissions go for the entire bucket, not for individual objects (JSON)
- Access Control Lists: Applied at object level, can define which accounts or groups are granted access, fine grained control for objects in the S3 bucket
- S3 Access Logs: Not enabled by default but logs all requests made to the bucket

S3 Encryption

- Encryption in Transit: Using SSL/TLS, HTTPS
- Encryption at Rest: Server-Side Encryption
    - SSE-S3: S3 Managed Keys, using AES 256-bit encryption (Default)
    - SSE-KMS: AWS Key Management Service-manged keys

        - You can choose an AWS Managed Key or a Customer Managed Key
    - SSE-C: Customer-provided keys
- Encryption at Rest: Client-Side Encryption
    - You encrypt the files yourself before uploading into S3
- You can enforce encryption with a bucket policy
    - Can deny requests that do not include the **x-amz-server-side-encryption** parameter in the request header and deny requests that do not use **aws:SecureTransport** to enforce HTTPS/SSL

Static Site Hosting and CORS

- Enable Static Site Hosting after creating bucket and specify index and error document
- Can edit COR policy in S3
- CORS (Cross Origin Resource Sharing): Can be used to allow resources in one S3 bucket to access resources in another S3 bucket

CloudFront

- Content Delivery Network: System of distributed servers which deliver webpages and other web contents
- Origin: Origin of all files that distribution will serve (S3, EC2, ELB or Route53)
- Edge Locations (Over 200 locations): Caches your webpage at that location
- CloudFront Origin: Origins of all files (Can be S3 bucket, Ec2 instance, ELB, Route53)
- CloudFront Distribution: Name given to the Origin and configuration settings
- Time to Live: Time item lives in cache (Default is 1 day)
- S3 Transfer Acceleration: Enables fast and secure transfer of files over long distances between S3 bucket and user
    - If many people want to upload to an S3 bucket located far way, they can use this

CloudFront with Origin Access Identity (OAI)

- Restricting access to only CloudFront
- Need to restrict access on S3 bucket
- Origin Access Identity is a special CloudFront user that can access the files in the bucket and serve to others
- Special feature in Amazon CloudFront designed to securely restrict access to content stored in an Amazon S3 bucket, ensuring that users can only access the content through the CloudFront distribution and not directly using the S3 bucket's public URL

CloudFront AllowedMethods

HTTP Methods:

Read Only:

- GET: Read data
- HEAD: Inspect resource headers, reads web page’s header
- OPTIONS: Find out which other HTTP methods are supported

Write Method:

- PUT: Send data to create or **replace** a resource
- PATCH: Partially **modify** a resource
- POST: Insert data, used to **create** or update a resource
- DELETE

There are 3 options to choose from, you need to determine what permissions you want to grant your users

Certificate Management

- Enables secure connections using HTTPS, certificates must be created in the us-east-1 Region when using ACM with CloudFront

Athena

- Interactive Query status
- Enables you to run standard SQL queries on data stored in S3
- Nothing to provision (serverless)
- Useful for querying log files, cost analysis, generating business reports, and queries on click-stream data (detailed, chronological log of a user's activity on a website or application)
- Steps:
    1. Point Athena to the data you want to query in S3
    2. Define a table schema
    3. Query your data using SQL

Questions

1. A developer works with applications in her AWS account that use Amazon S3 to store sensitive data. To enhance security, the developer wants to ensure that all S3 buckets in her applications are not publicly accessible. Which of the following actions should the developer take to meet this requirement? Choose the best option.
    A. Enable block public access settings at the account level to apply to all current and future S3 buckets in the account.
    B. Enable block public access settings at the bucket level and use an IAM policy to deny any public access.
    C. Enable block public access settings at the bucket level for each S3 bucket in the applications.
    D. Enable block public access settings at the account level and use IAM roles to manage access to specific S3 buckets.

    Answer: A

2.

3. What is the minimum file size allowed in an S3 bucket?
   A. 0 Bytes
   B. 1 KB
   C. 256 bytes
   D. 1 byte

Answer: A. Individual Amazon S3 objects can range in size from a minimum of 0 bytes to a maximum of 5 terabytes.

Serverless Computing

Serverless 101

- AWS manages capacity provisioning, patching, auto scaling, high availability
- Gives you competitive advantage: Speed to market, super scalable, lower costs, focus on application
- Serverless Services: Lambda, SQS, SNS, API Gateway, DynamoDB, S3

Lambda

- Serveless Compute with Enterprise Features
- Charged based on number of requests (first million per month are free), duration (charged by millisecond), and amount of memory used by function (by GB-second)
- Event-Driven Architecture: Can be automatically triggered by other services or events (like changes made to S3 bucket or DynamoDB table) or by user requests using API Gateway

API Gateway

- API: Application Programming Interfaces, we use them to interact with web applications, and applications use APIs to communicate with each other
- API Gateway publishes, maintains, and monitors APIs and supports **RESTful** APIs (stateless, serverless workloads) and **Websockets** APIs (real-time, two-way, stateful communication like chat apps)
- RESTful APIs: Representation State Transfer (REST)
- A WebSocket API enables two-way, persistent communication between a client (like a web browser) and a server over a single, long-lived connection.
- API Gateway allows you to connect to applications running on Lambda, EC2, Beanstalk, DynamoDB, Kinesis, etc.
- Serverless, allows throttling, integrates with CloudWatch (logs API calls, latencies, and errors)

Lambda Concurrent Executions Limit

- Safety feature to limit the number of concurrent executions across all functions in a given region per account (default 1000 per region)
- Reserved concurrency guarantees that a set number of executions will always be available for critical functions
- Hitting the limit will give a **429** status code
- Contact AWS Support to raise the limit if this is an issue

Lambda Versioning

- Latest version is always called $LATEST
- Can have aliases to reference older versions of your function

Lambda and VPC

- Can enable Lambda to access resources in a private VPC by providing config information to the function
- Lambda configures an ENI using an IP from the private subnet CIDR range

Example Serverless Architectures

- Asynchronous: An event or message might trigger an action, but no response is expected or required (Fire and forget)
- Loosely Coupled: Components operate and scale independently of each other
- Banking Application:Customer Withdraws money -> API Gateway-> Withdraw Lambda function -> DynamoDB -> EventBridge -> Overdraft increase Lambda function -> SQS
  -> Overdraft Notification Lambda function -> SNS
- Image Processing: Customer Uploads Image to Bucket -> Image Processing Lambda function -> Stores image in S3 bucket <- Served by CloudFront

Step Functions

- Visual interface for serverless applications
- Manages the logic of the application, including sequencing, error handling, and retry logic
- Helps debug applications, see where things went wrong

Step Functions Workflows

Workflow Structure Types

- Sequential Workflow: Steps happen one after another (output of one step is the input to another)
- Parallel Workflow: Tasks happen in parallel
- Branching Workflow: Basically if statements depending on outputs that routes the flow

Workflow Execution Types

- Standard Workflows: Long-running, durable and auditable workflows that run for up to 1 year, tasks are never executed more than once, non-idempotent actions (always causes a change in state), tasks take a while to complete like verification or credit check
- Express Workflows: High volume, event-processing-type workloads, might be run more than once or multiple concurrent executions, idempotent actions (identical requests executed sequentially have no side effects)
    - Synchronous: Begins the workflow, waits until it completes, returns result
    - Asynchronous: Begins the workflow, confirms workflow has started, result can be found in CloudWatch Logs (doesn't wait)

Ephemeral and Persistent Data Storage Patterns

- Lambda functions are stateless: Can't permanently store any data in the function
- /tmp: Temporary storage in the execution environment of lambda, like a cache file system, 512MB - 5 GB
- Layers: Lambda libraries and SDKs
- Persistent Storage Options:
    - S3: Store and retrieve objects, not a file system, no append only uploading new version
    - EFS: Elastic File System, shared file system and can be dynamically updated, needs to be mounted by the function, must be in same VPC

Lambda Environment Variables and Parameters

- Environment Variables: Allows you to adjust functions behavior without changing your code, are key value pairs
    - Examples: Referencing S3 resources, SNS Topic, or DynamoDB Table

- Configurable Parameters: Memory, Ephemeral Storage, Function timeout, triggers, permissions, function URL, tags, VPC, monitoring tools, reserved concurrency, filesystem

Lambda Event Lifecycles and Errors

- Invocation Lifecycle:
    - Synchronous or Asynchronous
- Lambda Retries: Lambda automatically makes 2 retries, waits 1 minute and then 2 minutes for retries
- Dead-Letter Queues: Save failed invocations for further processing, associated with a particular version of a function, can be event source (handle failures only)
    - We can use SQS or SNS
- Lambda Destinations: Configure lambda to send invocation records to another service (Example: If success, send to SQS, if failure send to SNS)

Lambda Deployment Packaging Options

- Deployment Packages: Zip file containing application code and dependencies
- Packages greater than 50 MB: Will need to upload the zip file to S3 in the region where you would like to create the function
- Lambda Layers: Zip file archive referenced by function containing function dependencies, best practice to use

Lambda Performance Tuning Best Practices

- Memory and CPU Capacity: You can only add more memory, adding more memory adds CPU capacity
- Execution Environment:
    - 1. Download Code: During first invocation, lambda creates execution environment
    - 2. Configuration: Memory, runtime, configuration
    - 3. Static Initialization: Runs function static initialization code, imports libraries and dependencies
    - 4. Executes function
- Optimizing Static Initialization:
    - 1. Amount of code that needs to run
    - 2. Function Package Size
    - 3. Performance
    - Avoid importing entire library

Advanced API Gateway

- You can import your own API with definition files
    - OpenAPI (formerly Swagger) is supported
    - Can use OpenAPI definition file to create a new API or update an existing API

- Legacy Protocols
    - SOAP (Simple Object Access Protocol) returns in XML
    - Can configure API Gateway as a SOAP web service pass-through
    - Can also use API Gateway to transform XML response to JSON

API Mock Endpoints

- Allows developers to create, test, and debug software
- Simulate the responses and behaviors that you would expect from an API
- You define response, status code, and message

API Gateway Stages for Testing Deployed Code

- API Gateway Stage: Logical reference that references a lifecycle state of API (dev, prod, v3, etc.)
- Stage Variables: Key-Value pairs that act like environment variables, can be used to reference a specific backend point like a lambda function

API Response and Request Transformations

- **Request Transformations**: Gateway can modify request parameters from frontend to backend
    - Can modify the header, query string, or request path of an API request
- **Response Transformations**: Gateway can modify response parameters from backend to frontend
    - Can modify header or status code of an API response
- Parameter Mapping: Used to modify API requests and response in HTTP APIs, you specify what you want changed and how

API Gateway Caching and Throttling

- Caching of endpoint response (default TTL is 300 seconds)
- Throttling limits rate to 10,000 request per second per region, max 5,000 requests across all APIs per region, **429** error message if exceeded

X-Ray

- Helps developers analyze and debug distributed applications, provides visualization of application's underlying components
- Tracks Latency, HTTP Status Codes, Errors
- Need to install agent on instance, and configure, and the SDK gathers information
    - Agent and SDK (also called daemon) are required, SDK sends data to daemon
- Kind of like a lower leveled version of Step Functions, Step Functions is high level where you can see workflow logic (like A triggers lambda function B etc.) and X-Ray shows more indepth of flows and error codes

X-Ray Configuration

- Install the daemon (agent), then set your annotations (additional information about requests in key-value pairs) which can be filtered in searching

Question

1. Which of the following represents the features of Lambda that are used for error handling?
    a. SQS and dead-letter-queues
    b. Layers and triggers
    c. Dead-letter-queues and destinations
    d. Triggers and SQS

Explanation:

- Dead-letter queues and destinations (Correct Answer): Used to handle failed asynchronous Lambda invocations — DLQs store failed events for later review, while Destinations route success or failure results to other services like SQS, SNS, Lambda, or EventBridge.
- SQS and dead-letter queues: SQS can serve as a DLQ target, but it isn't itself an error handling feature built into Lambda.
- Layers and triggers: Layers are for sharing code or libraries across functions, and triggers are used to automatically invoke Lambda functions — neither handle errors.
- Triggers and SQS: Triggers start Lambda executions, and SQS can be an event source, but neither are used for managing or handling errors.

DynamoDB

DynamoDB 101

- Low latency NoSQL Database (Flexible)
- Supports key-value data models, and documents: JSON, HTML, XML
- Automatically scale, serverless
- Features
    - Data is stored in SSD storage (fast)
    - Spread across 3 geographically distinct data centers
    - Eventually Consistent Reads (default) or Strongly Consistent Reads
        - Eventually Consistent Reads: All copies of data are reached **within a second** (Good for read performance)
        - Strongly Consistent Reads: Reflects all successful writes **instantly** (Good for read consistency)
        - ACID Transactions**:** DynamoDB Transactions provide the ability to perform Atomic, Consistent, Isolated, and Durable transactions (**All or nothing** operation)
- Items: Row in a table (made up of attributes/column)
- Primary Keys

    - Partition Key: Based on a unique attribute (customerId, productId, etc)
    - Composite Key (Partition key + Sort Key): Unique combination where partition key can be the same (like user_id and timestamp)

DynamoDB Access Control

- Use IAM policy to for access to specific tables
- Can restrict user access by comparing to partition key
    - dynamodb:LeadingKeys: Allows users to access only the items where the partition key value matches their User_ID

DynamoDB Secondary Indexes

- Can query based on an attribute that is not the primary key using global and local secondary indexes (faster)
- Local Secondary Index:
    - Same partition key but different sort key
    - Must be created when creating the table
- Global Secondary Index:
    - Different partition key and sort key
    - Can be created whenever
- A secondary index provides an alternate key structure to enable fast sorting and searching (querying) on attributes other than the table's primary key.

DynamoDB Query and Scan

- Query: Finds items in a table based on primary key and a distinct value to search for
    - Can be refined by sort key
    - ProjectionExpression parameter limits only specific attributes to return
    - ScanIndexForward=False will reverse the order returned (For queries only even though it has scan in name)
- Scan: Examines every item in the table
    - ProjectionExpression works here too
    - Can filter based on other attributes (not only primary key) and can use conditionals
- Improving Performance:
    - Setting a smaller page size
    - Isolate scan to specific tables
    - Parallel scans
    - Use Query if possible

DynamoDB API Calls

- create-table, put-item, get-item, update-item, update-table, list-tables, describe-table, scan, query, delete-item, delete-table
- CLI commands are making calls to DynamoDB API

- Correct IAM permissions are required

DynamoDB Provisioned Throughput

- Measured in Capacity Units
- Write Capacity Units: 1 write capacity unit = 1x 1kb/s
- Read Capacity Units = 1x strongly consist read of 4kb/s or 2x eventually consistent reads of 4kb/second (default)
- Example: 5 Read capacity Units and 5 Write capacity Units
    - 5 x 4kb strongly consistent reads = 20kb/s
    - 5x2 x4kb eventually consistent reads = 40kb/s
    - 5 x 1kb writes = 5kb/s
- Example 2: You need to read 80 items/second, each item is 3kb, require strongly consistent, how many read capacity units?
    - We assume each item is 4kb (round up) so 1 unit, 80 items => 80 read capacity units
    - For eventually consistent, /2 at the end so 40 capacity units
- Example 3: You want to write 100 items/s, each item is 512 bytes in size, how many write capacity units do you need
    - Size of each item / 1kb = 512 bytes = 1024 bytes = 0.5, round up => 1
    - 1 x 100 items/s = 100

DynamoDB On-Demand Capacity

- Charges apply for reading, writing, and storing data
- DynamoDB scales up and down based on activity
- On-Demand Capacity vs Provisioned Capacity

DynamoDB Accelerator (DAX)

- Fully managed, clustered in-memory cache
- Up to 10x improvement in read performance (Great for read-heavy workloads)
- Item is added to the cache as well as the backend
- If cache miss, DAX performs an eventually consistent GetItem operation against DynamoDB and returns the result of API call
- If application requires strongly consistent reads, this isn't helpful

DynamoDB Time to Live (TTL)

- Expiry time for data
- Expressed in Epoch Time (Unix Time)
- Item will be deleted within 48 hours of TTL

DynamoDB Streams

- **Records modifications** (in the time order it was performed) into logs and stored for 24 hours
- Good for auditing, archiving, or triggering an event

Provisioned Throughput Exceeded and Exponential Backoff

- If you make too many requests, you could get ProvisionedThroughputExceededException
- AWS SDK will automatically retry the requests
- Exponential Backoff: Progressively longer waits between consecutive retries

Additional Information

- dynamodb:LeadingKeys allows users to access only the items where the partition key value matches their User_ID
- Popular CLI commands: CreateTable, GetItem, PutItem, UpdateItem, UpdateTable, ListTable

Question

Your application is storing customer order data in a DynamoDB table. You need to run a query to find all the orders placed by a specific customer in the last month. Which attributes would you use in your query?

A. A composite primary key made up of the CustomerName and OrderDate attributes
B. A partition key of CustomerID and a sort key of OrderDate
C. A partition key of OrderDate and a sort key of CustomerName
D. A partition key of OrderDate and a sort key of CustomerID

Answer: B, A is possible if Customer Names are unique but bad practice anyways

KMS And Encryption

KMS101

- Managed service to create and control encryption keys
- Use whenever dealing with sensitive information (Customer data, passwords, secrets, credentials, etc.)
- Integrates with other services (S3, RDS, DynamoDB, Lambda, EBS, EFS, Cloudtrail, Developer tools, etc.)

CMK Summary

- Customer Master Key: Encrypt/Decrypt data up to 4KB

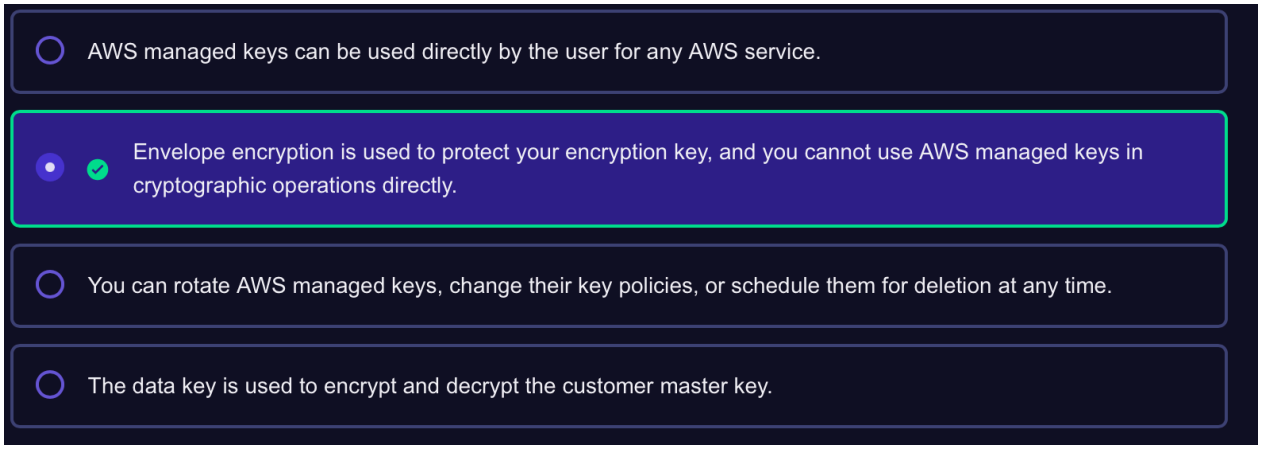
- Used to Generate/Encrypt/Decrypt the Data Key
    - Data Key: Used to Encrypt/Decrypt your data
- Envelope Encryption:
    - Encrypting the key that encrypts our data
    - CMK is used to encrypt the data (envelope) key
- Features:
    - Application can refer to the alias
    - Contains creation date and time, and description
    - Has a key state (Enabled, Disabled, Pending Deletion, Unavailable)
    - Can be customer-provided or AWS-provided
    - Can never be exported
- Steps:
    - Set up CMK: Create an alias, description, and key material option (KMS, External, Custom key store)
    - Key Administrative Permissions: IAM users and roles that can administer (but not use) the key through the API (Choose users, roles)
    - Key Usage Permissions: IAM Users and roles that can use the key to encrypt/decrypt data
- AWS-Managed CMK: AWS-provided and AWS-managed CMK. Used on your behalf with the AWS services integrated with KMS
- Customer-Managed CMK: You create, own and manage yourself
- Data Key: Encryption keys that you can use to encrypt data. You can use a CMK to generate, encrypt, and decrypt data keys
- CLI commands:
    - aws kms encrypt: Encrypt plaintext into ciphertext by using a CMK
    - aws kms decrypt: Decrypts ciphertext
    - aws kms re-encrypt: Decrypts ciphertext and then re-encrypts it entirely within KMS (when you change the CMK or manually rotate the CMK)
    - aws kms enable-key-rotation (Can also enable in the console after creating it and saves previous versions so you can still decrypt files that were previously encrypted, rotates on a 365-day basis)
    - aws kms generate-data-key: Uses the CMK to generate a data key to encrypt data > 4 KB
- Note: CMK Keys are now called KMS Keys (since 2021)

Certificate Management

- Create and manage public and private SSL/TLS certificates (Enables secure connections)
- Can be used with Elastic Load Balancing, CloudFront, API Gateway, and web applications
- When using ACM with CloudFront, certificate must be created in us-east-1 region

Question

1. **What are the key characteristics of AWS managed keys?**



Explanation: When you encrypt your data, your data is protected, but you have to protect your encryption key. One strategy is to encrypt it. Envelope encryption is the practice of encrypting plaintext data with a data key and then encrypting the data key under another key. You can view AWS managed keys and their key policies in your account and audit their use in AWS CloudTrail logs. However, you cannot manage, rotate, or change their key policies. AWS managed keys are created and managed by AWS for specific services, such as Amazon S3, Amazon EBS, and Amazon RDS. These services use AWS-managed keys to encrypt your data, but you cannot use them directly yourself.

Other AWS Services

SQS 101

- Message queue service
- Enables web service applications to queue messages that one component in the application generates for another component to consume
- Messages are stored in SQS, could be jobs stored as messages
- **Pull based service (You pull jobs from SQS)**
- Visibility Timeout
    - Amount of time a server gets to process the message
    - **Decouples** components of an application so they run independently
- Up to 256 KB per message
- Is a buffer between components

SQS Queue Types

- Standard queues: Best-effort ordering
    - Guarantees that a message is delivered at least once
    - Can duplicate and be slightly out of order

    - No limit on transactions
- FIFO queues (Ordering is preserved)
    - 300 transactions per second limit

SQS Settings

- **Visibility Timeout**: Amount of time that the message is invisible in the queue after a reader picks up that message
    - Default is 30 seconds, Max is 12 hours
- **Short Polling**: Returns response immediately even if message queue being polled is empty (still gets charged for empty responses)
- **Long Polling**: Periodically polls the queue, no response until a message arrives in the queue (preferable generally)
- Polling is like when lambda sends HTTP response to SQS to see if anything is in queue

SQS Delay Queue and Large Messages

- Can postpone delivery of new messages to a queue, maximum is 900 seconds
    - Affects delay of messages already in queue for FIFO only
    - Delay in processing may be necessary
- Use S3 to store large messages (256kb to 2GB)
    - Requires SQS Extended Client Library and SDK
    - Use the Extended Library to manage large messages (automatically send large files to S3)

SNS 101

- Push Notifications to devices
- SMS text messages or email to SQS or any HTTP endpoints
- Trigger Lambda functions to process information in the message or send to another AWS Service
- Pub-Sub Model (Publish-Subscribe)
    - Applications publish or push messages to a topic
    - Subscribers receive messages from a topic
- Durable storage

SES vs SNS

- Simple Email Service
    - Send and receive emails delivered to an S3 bucket
    - Can trigger lambda functions and SNS notifications
    - Good for automated emails
- SNS: Must subscribe to the topic to receive notifications and cant receive emails

Kinesis 101

- A family of services that enables you to **collect, process, and analyze** streaming data in real time (data usually in small sizes in kb) Update: As of late 2025, Kinesis supports up to 10MB
- Kinesis Streams: Allows you to build custom applications that process data in real time
    - Made up of shards
        - Each shard is a sequence of one or more data records; the number of shards determines the data capacity
    - Data Streams and Video Streams
        - Each record in the stream has its own unique number
- Kinesis Data Firehose: Capture, transform, and load data streams into AWS data stores to enable near real-time analytics
- Kinesis Data Analytics: Analyze, query, and transform streamed data in real time using standard SQL
    - Happens after Firehose or Streams

Kinesis Shards and Consumers

- Kineses Client Library load balances according to the number of shards
- Ensure that the number of instances (consumers) doesn't exceed the number of shards
- One instance can handle multiple shards, CPU utilization should drive quantity of consumer instances

Elastic Beanstalk 101

- **Deploy and scale web applications**
- You provide code, beanstalk does the rest
    - Handles infrastructure (load balancing, auto scaling, application health balancing)
    - Application platform: Installation and management of application stack, patching and updates

Updating Elastic Beanstalk

- **All at once**: Deploys to all hosts concurrently
    - There will be total outage when a new version releases
    - Pretty much only for test environments
- **Rolling**: Deploys the new version in batches
    - No outage but capacity is reduced when update is taking place, not ideal for performance sensitive systems
- **Rolling with Additional Batch**: Launches an additional batch of instances, then deploys the new version in batches
    - Can maintain full capacity during updates, pretty good options except for roll backs which take some time
- **Immutable**: Deploys the new version to a fresh group of instances before deleting the old instances

    - Terminates old instances after the new instances pass their health checks
    - Traffic Splitting: Installs the new version on a new set of instances like an immutable deployment, then forwards a percentage of incoming client traffic to the new application version for evaluation
        - Enables **Canary Testing**: Can direct a small percentage of traffic to the new instances for a period for testing

Advanced Elastic Beanstalk

- Amazon Linux 1 (Retired in 2022, probably won't be on exam)
    - Config file in yaml or json with a .config ending inside a folder called .ebextensions
- Amazon Linux 2
    - Buildfile, Procfile, and platform hooks for configuration
        - **Buildfile**: Commands that run for short periods (like shell script)
        - **Procfile**: Long running application processes
        - **Platform hooks**: Custom scripts or executable files that you want to run at a chosen stage, stored in dedicated directories (.platform folder instead of .ebextensions)
- Amazon Linux 2023
    - Uses Buildfile, Procfile, and platform hooks like Amazon Linux 2
    - Platform hooks are simplified and support more modern lifecycle stages

RDS & Elastic Beanstalk

- Deploy RDS inside of Beanstalk environment or outside of it
    - Environment termination also terminates RDS if its inside of Beanstalk (not great for production)
    - Deploying outside of beanstalk requires an additional security group to be added to environment's auto scaling group and a connection string to application servers

Migrating Applications to Elastic Beanstalk

- Migration Assistant Tool: Windows Web Application Migration Assistant that enables you to migrate a **.net application** from windows servers to elastic beanstalk (open source)

Developer Theory

CI/CD

- Make small changes and automate everything (code integration, build, test, deployment)
    - Allows you to make thousands of changes per day, integrating or merging the code frequently
    - Test each code change and catch bugs while they are small and simple to fix

    - Continuous Integration (Think CodeCommit)
    - Continuous Delivery (Think CodeBuild and CodeDeploy)
    - Continuous Deployment (Think CodePipeline)
- Shared code repository, automated build, automated test
- Continuous Delivery: Manual decision to deploy code
- Continuous Deployment: Automated deployment of code once its ready
- **CodeCommit**: Source control service (**Like GitHub**)
- **CodeBuild**: Complies source code, runs tests, produces packages (**Like GitHub Actions Runners**)
- **CodeDeploy**: Automates code deployments to any instance like EC2 or lambda (**Like GitHub Actions Deployment Steps**)
- CodePipeline: End-to-end solution, build, test, and deploy every code change **(Like Github Actions Workflows**)

CodeCommit 101

- Source code, binaries, libraries, images
- Manages updates from multiple sources, enables collaboration, tracks and manages code changes, maintains version history
- Based on git

CodeDeploy 101

- Automated deployments lets you quickly release new features, avoid downtime, and avoid risks
- 2 Approaches
    - **In-Place deployment**, the application is stopped on each instance and the new release is installed (Rolling Update)
        - Revision (application version)
        - Will need to re-deploy the previous version for rollback
    - **Blue/Green deployment**: New instances are provisioned and the new release is installed on the new instances. Blue represents active deployment, green is the new release

CodeDeploy AppSpec File

- **Defines the parameters** to be used during a deployment
- For EC2 and on-premises systems in YAML only
- YAML and JSON are supported for Lambda
- Version, os, files (location of application files), hooks (scripts that need to run at set points in lifecycle)
    - Scripts you might run are unzip files, run tests, deal with load balancing
- AppSpec.yml has to be in root directory

CodeDeploy Lifecycle Event Hooks

- Like Jobs or Steps in Github Workflows
- Hooks run in specific order (run order)
    - BeforeInstall, AfterInstall, ApplicationStart, ValidateService, etc.
- 3 Phases
    - De-register instances from load balancer
        - BeforeBlockTraffic: Any tasks before deregistering
        - BlockTraffic: De-register instances from load balancer
        - AfterBlockTraffic: Any tasks after deregistering
    - Application Deployment Phase
        - ApplicationStop: Gracefully stopping application
        - DownloadBundle
        - BeforeInstall: Pre-installation scripts like backing up or decrypting files
        - Install
        - AfterInstall: Post-install scripts like configuration
        - ApplicationStart
        - ValidateService
    - Re-register instances with load balancer
        - BeforeAllowTraffic
        - AllowTraffic
        - AfterAllowTraffic

CodePipeline 101

- Can be configured to run every time there is a change in code
- Integrates with CodeCommit, CodeBuild, CodeDeploy, GitHub, Jenkins, Elastic Beanstalk, CloudFormation, Lambda, ECS

Code Artifacts

- Artifact repository that allows storing, publishing, and sharing artifacts
- Can be used by all developers to obtain correct versions of software packages required for projects
- Integrates with public repositories
- Create a CodeArtifact Domain, create a repository in domain, and upstream repository
    - Domain connects to public repository
    - Allows you to pull packages from an external public repository

Elastic Container Service

- Containers are like a **virtual operating environment** with everything software needs to run
- Applications are created using independent stateless components or microservices
- Can use Docker (Linux) or Windows Containers
- ECS allows you to deploy and scale containerized workloads

- Can run containers on Clusters of Virtual Machines, or Fargate for Serverless, or EC2

CloudFormation

- Manage, configure, and provision AWS infrastructure as code (YAML or JSON)
- Allows consistency, quick and efficient, version control, rolling back, manage updates
- Create YAML or JSON file, upload it to CloudFormation using S3, CloudFormation reads template and makes API calls, resulting set of resources is called a stack
- Can use parameters, use code outside of main template, create mappings based on region
- Structure:
    - Parameters: Input custom values
    - Conditions: Provision resources based on environment
    - Resources: Only mandatory section, AWS resources to create
    - Mappings: Custom Mappings like Region: AMI
    - Transforms: Reference code located in S3

Serverless Application Model

- **Extension for CloudFormation** for Serverless applications
- SAM CLI
    - sam package: Packages application and uploads to S3
    - sam deploy: Deploys serverless app using S3

CloudFormation Nested Stacks

- Can use **outputs of one stack for an input of another stack**
- Good for frequently used configs like load balancers
- Create a CloudFormation template, store in S3, and reference it in Resources section as a stack

Cloud Development Kit (CDK)

- Open-source software development framework, allows you to build applications, define, and deploy resources **using programming language** of your choice
- Can transform a script in another language to CloudFormation template
- App: Container of one or more stacks
- Stack: Unit of deployment containing one or more construct
- Construct: Defines an AWS resource
- cdk init -> npm run build -> cdk synth -> cdk deploy
    - Creates and deploys CloudFormation template

Amplify

- Set of tools and services designed to make it easy to develop **web and mobile applications** on AWS

- Takes care of developing a reliable backend, integrated with Cognito, S3, Lambda, and API Gateway
- **Amplify Hosting**: Fully managed web hosting service with CI/CD functionality and integrates with code repository, does web applications and static website hosting
- **Amplify Studio**: Visual interface to create frontend UI and backend

Advanced IAM

Web Identity Federation

- Simplifies authentication and authorization for web applications, allows users access to AWS Resources after authentication with a web based identity provider like Facebook, Amazon, or Google
- Amazon Cognito: Acts as an identity broker that provides web ID federation, including sign-up and sign-in functionality, recommended for all mobile applications, provides temporary credentials which map to an IAM role

Cognito User Pools

- User Pools: User directories used to manage **sign-up and sign-in functionality** for mobile and web applications
- Identity Pools: Enable you to **provide temporary AWS credentials** enabling access to AWS services
- Cognito Push Synchronization uses SNS to update records to other devices when information changes
- User -> Logs into Facebook where a User Pool brokers the sign-in and sign-up functionality with Facebook -> Facebook returns JWT (JSON Web Token)Token -> Exchanged for AWS Credentials by Identity Pool -> Mapped to IAM Role -> Gives access to AWS Resources

Inline Policies vs Managed Policies vs Customer Managed Policies

- Managed Policies: IAM Policy created and Administered by AWS
- Customer Managed Policies: Standalone policy that you create and administer inside your own AWS account
- Inline Policies: Cannot be re-used, are attached to an entity

STS AssumeRoleWithWebIdentity

- Security Token Service API Call, Returns temporary security credentials for users authenticated by a mobile or web application or using a web ID provider like Amazon, Facebook, Google, etc.
- User-> Authenticates with Facebook, returns JWT Token to application, application calls assume-role-with-web-identity passing in JWT Token, Token is exchanged for AWS Credentials, which lets you access resources

Configuring Cross Account Access

- Delegating access to resources in different AWS accounts that you own, allows sharing of resources
- Create an IAM role to allow access and grant permissions to users in a different account
- Can switch roles within the AWS management console

Monitoring

CloudWatch 101

- Monitoring (performance and health) service (pretty much any service)
    - **CloudWatch**: Default EC2 host-level metrics: CPU, network, disk, status check
- Can define your own custom metrics with **CloudWatch Agent**
    - If you install CloudWatch Agents, you can collect operating system metrics and send them to CloudWatch in EC2 (Memory usage, processes, free disk space, CPU idle time, etc.)
- **CloudWatch Logs**
- All EC2 instances send key health and performance metrics to CloudWatch (stored indefinitely by default)
- By default, EC2 sends metric data to CloudWatch in 5-minute intervals
- **CloudWatch Alarms**: Can include EC2 CPU utilization, latency, charges on AWS Bill, etc.
    - Can set appropriate thresholds
- Can create CloudWatch Dashboards and have widgets

CloudWatch Concepts

- Metrics: Variable to monitor
- Namespace: Container for metrics (EC2 uses AWS/EC2 namespace), can create own namespace
- Dimensions: Filter
    - Can provide aggregate data across all instances
- Dashboard: Custom view

CloudWatch vs CloudTrail

- CloudTrail: Records user activity in the AWS account **(API Activity)**
    - Related to the **creation, modification, or deletion** of resources
    - Can view the last 90 days of activity
    - API Calls for AWS Account to an S3 bucket
    - Can be integrated with CloudWatch Logs
- CloudWatch: Performance and Metrics, CloudWatch Logs, CloudWatch Alarms **(For performance and health)**
- Monitor Performance: CloudWatch

- Audit log of user activity: CloudTrail

CloudWatch Actions

- Actions: Allow publishing, monitoring, and alerts for a variety of metrics
- PutMetricData: Publishes metric data points
    - aws cloudwatch put-metric-data –metric-name PageViewCount –namespace MyService –value 25 –timestamp 2022-01-10T12:00:00.000Z
- PutMetricAlarm: Creates an alarm associated with a metric to alert you if a threshold has been reached
    - aws cloudwatch put-metric-alarm –alarm-name PageViewMon –alarm-description “PageView Monitor” –metric-name PageViewCount –namespace MyService –statistic Average –period 300 –threshold 50 –comparison-operator GreaterThanThreshold –evaluation-periods 1

CloudWatch Logs Insights

- Interactive query and analysis for data stored in CloudWatch Logs
- Can generate visualizations

HTTP Error Codes and SDK Exceptions

- Client Side Errors: 4xx
    - 400: Access Denied
    - 403: Missing Authentication Token
    - 404: Malformed Query String
- Server Side Errors: 5xx (5 looks like an S as in Server)
    - 500: Internal Failure
    - 503: Service Unavailable (Temporary failure of the server)
- BatchGetItem: Returns details of one or more items from a table
    - ValidationException (Too many items)
    - UnprocessedKeys: Some items were not successfully processed (Reduce request size)
    - ProvisionedThroughputExceededException
- BatchWriteItem: Puts or deletes one or more items in one or more DynamoDB tables
    - UnprocessedItems: Some operations failed, will need to retry the processed items
    - ProvisionedThroughputExceededException

EventBridge 101

- Event-driven architecture (change in state)
- Rules can run on a schedule
- CloudWatch Events is basically the same, but with fewer features, use EventBridge recommended

- Receives event relating to state changes, can use EventBridge to create rules that take actions based on the event it receives (like send SNS), EventBridge and CloudWatch Events use same underlying technology

Additional Information

- AWS AppSync simplifies application development by giving you the ability to create a flexible API to securely access, manipulate, and combine data from one or more data sources. AWS AppSync is a managed service that uses GraphQL to help applications get the exact data that they need. You can use AWS AppSync to build scalable applications that require real-time updates on a range of data sources, including Amazon DynamoDB
- Lazy loading is a caching strategy in which a record does not load until the record is needed. When you implement lazy loading, the application first checks the cache for a record. If the record is not present, the application retrieves the record from the database and stores the record in the cache

Feature	**Amazon RDS**	**Amazon DynamoDB**
Type	Relational (SQL)	Non-Relational (NoSQL)
Data Model	Tables with fixed rows and columns	Key-value pairs or JSON documents
Schema	Rigid (must define columns upfront)	Flexible (schemaless)
Scaling	Vertical (bigger servers)	Horizontal (more servers)
Queries	Complex (JOINs, aggregations)	Simple (fast lookups by Key)

Management	Managed Servers (you choose size)	Serverless (no servers to manage)